CSC322 Project 2

Search Engine — Class Diagram & Design Documentation

Document representation, query representation, matching/ranking and API facade, designed as abstract data types with supporting design patterns.

1. Overview

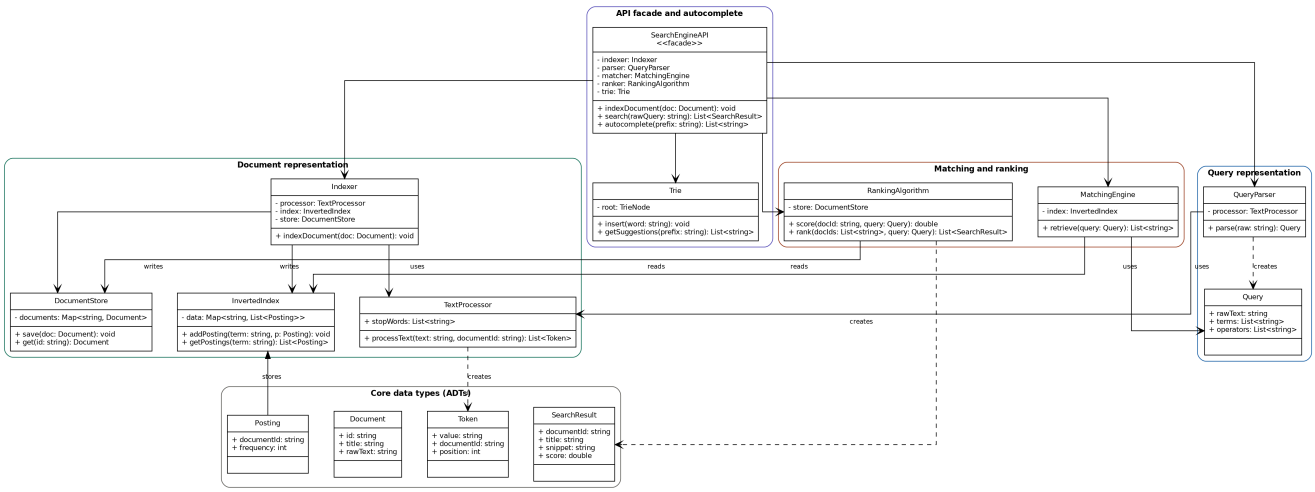
This document works from the assignment brief's abstraction: documents are already available in a repository (no crawler is required), and the system's job is to build a **document representation** (the indexer), a **query representation** (the query parser), and a **matching function** that compares the two and produces ranked **results**. Two independent pipelines share one underlying storage structure:

- **Indexing pipeline** (runs when a document is added): Document → text processing → Indexer → storage (InvertedIndex + DocumentStore).
- **Query pipeline** (runs when a user searches): Query → text processing → MatchingEngine → RankingAlgorithm → SearchResult list.

The whole system is exposed to client applications through a single facade class, `SearchEngineAPI`, matching the requirement that the deliverable is an API.

2. Class diagram

Boxes are grouped by module (core ADTs, document representation, query representation, matching & ranking, API facade). Solid arrows are associations/uses; dashed arrows are "creates" dependencies; the open-diamond arrow is an aggregation (InvertedIndex stores many Posting objects).



3. Class descriptions

3.1 Core data types

Small, dependency-free ADTs used across every module. None of these contain business logic — they only hold data.

Token (core)

A single normalised word produced from a document or a query, plus where it came from.

Member	Description
`value: string`	The normalised term itself (lowercased, stemmed if you choose to stem).
`documentId: string`	Which document this token was extracted from.
`position: int`	Word position within the document — needed only if you implement phrase search.

Posting (core)

One entry in a term's postings list: which document contains the term, and how often.

Member	Description
`documentId: string`	The document this posting refers to.
`frequency: int`	How many times the term occurs in that document — the raw input to TF-IDF/BM25 scoring.

Document (core)

An in-memory representation of one document after extraction, before indexing.

Member	Description
`id: string`	Unique identifier for the document (e.g. file path or generated id).
`title: string`	Display title, used in search results.
`rawText: string`	The clean extracted text, produced by your file-format extractor (e.g. Apache Tika).

SearchResult (core)

One ranked result returned to the client after a query is answered.

Member	Description
`documentId: string`	Identifies which document this result refers to.
`title: string`	Shown to the user in the results list.
`snippet: string`	A short excerpt of the content, usually built around a matched term.
`score: double`	The relevance score assigned by the RankingAlgorithm; results are sorted by this.

3.2 Document representation

Responsible for turning a raw Document into indexed, searchable data. Runs during the indexing pipeline.

TextProcessor (document representation)

Shared normalisation logic reused by both pipelines: strips stop words, lowercases, and tokenises raw text into a list of Token objects. Because both a document and a query need identical treatment, this class should not contain any document-only or query-only logic.

Member	Description
`stopWords: List<string>`	Fixed list of words excluded from indexing (prepositions, articles, etc., per the brief).
`processText(text, documentId): List<Token>`	Tokenises, lowercases and filters the given text, tagging each resulting Token with its source documentId.

Indexer (document representation)

Orchestrates the indexing pipeline for one document: calls TextProcessor, builds Posting entries, writes them into InvertedIndex, and caches the original content in DocumentStore so it never needs to be re-parsed just to be displayed.

Member	Description
`processor: TextProcessor`	Used to normalise the document's text before indexing.
`index: InvertedIndex`	Destination for the resulting postings.
`store: DocumentStore`	Destination for the original document content.
`indexDocument(doc): void`	Runs the full indexing pipeline for a single document.

InvertedIndex (document representation)

The core ADT of the whole project: a map from term to the list of documents containing it. Chosen so that term lookup — the operation your 0.01s response-time requirement depends on — is close to O(1).

Member	Description
`data: Map<string, List<Posting>>`	Term → postings list. Internal only; not exposed directly.
`addPosting(term, p): void`	Adds a posting to a term's list, creating the list if the term is new.
`getPostings(term): List<Posting>`	Returns every document (with frequency) containing the given term. This is what MatchingEngine calls.

DocumentStore (document representation)

A simple key-value cache of original document content, kept separate from the InvertedIndex so that displaying a result never requires re-opening or re-parsing the source file.

Member	Description
`documents: Map<string, Document>`	Id → full Document, built once at indexing time.
`save(doc): void`	Caches a document's content.
`get(id): Document`	Retrieves a document's content for display, by id.

3.3 Query representation

Turns raw query text into a structured representation the matching function can act on. This is the natural place to apply a parser generator, since a query with AND/OR/phrase syntax is a small formal language.

Query (query representation)

The structured result of parsing a raw query string. A simple version stores terms and operators as parallel lists; a more advanced version would store a parse tree (AND/OR/PHRASE nodes) generated directly by your grammar.

Member	Description
`rawText: string`	The original query text, kept for reference/logging.
`terms: List<string>`	Normalised terms extracted from the query.
`operators: List<string>`	Boolean operators between terms (AND, OR, NOT), if supported.

QueryParser (query representation)

Reads raw query text and produces a Query object. Internally reuses TextProcessor so that query terms are normalised in exactly the same way as indexed terms — otherwise nothing would ever match. In the full implementation this class is generated, or partly generated, by a parser generator (e.g. ANTLR) from a grammar you define.

Member	Description
`processor: TextProcessor`	Ensures query terms are normalised identically to indexed terms.
`parse(raw): Query`	Parses raw query text into a structured Query.

3.4 Matching and ranking

Finds candidate documents for a query, then orders them by relevance. This runs entirely against data already built by the indexing pipeline — it never touches the original files.

MatchingEngine (matching and ranking)

Given a parsed Query, looks up each term in InvertedIndex and combines the postings lists (intersection for AND, union for OR) to produce the candidate set of matching document ids.

Member	Description
`index: InvertedIndex`	Source of postings lists for each query term.

`retrieve(query): List<string>`	Returns the ids of every document judged relevant (before ranking).

RankingAlgorithm (matching and ranking)

Scores each candidate document's relevance to the query (e.g. TF-IDF or BM25) and sorts them, pulling title/snippet data from DocumentStore to build the final SearchResult list.

Member	Description
`store: DocumentStore`	Used to fetch title/content for building result snippets.
`score(docId, query): double`	Computes a relevance score for one document against the query.
`rank(docIds, query): List<SearchResult>`	Scores and sorts all candidates, returning them as SearchResult objects.

3.5 API facade and autocomplete

Trie (autocomplete)

A prefix tree built from every indexed term, used to answer the query-autocomplete requirement. Populated incrementally as the Indexer processes documents.

Member	Description
`root: TrieNode`	Root of the prefix tree (internal structure).
`insert(word): void`	Adds a term to the trie, called once per unique indexed term.
`getSuggestions(prefix): List<string>`	Returns all known terms that continue the given prefix.

SearchEngineAPI (API facade)

The single entry point exposed to client applications (wrapped by a thin Spring Boot @RestController). Hides every internal class above this line from the caller — a textbook Facade pattern application, and the direct answer to the brief's "solution should be in the form of an API" requirement.

Member	Description
`indexer / parser / matcher / ranker / trie`	Private references to one instance of each module above.
`indexDocument(doc): void`	Public entry point for adding a document to the search engine.
`search(rawQuery): List<SearchResult>`	Public entry point for answering a query end-to-end.
`autocomplete(prefix): List<string>`	Public entry point for the query-autocomplete feature.

4. Design patterns applied

Pattern	Where it is used	Why
Facade	SearchEngineAPI	Gives client applications one simple interface (indexDocument, search, autocomplete) while hiding the indexer, parser, matcher, ranker and trie behind it.
Strategy / Adapter	File-format text extraction (feeds Document)	Each input format (pdf, doc, docx, ppt, xls, txt, html, xml) gets its own extractor implementation behind one common "extract text" interface, so Indexer never needs to know which format it is handling.
Factory	Extractor selection	A small factory chooses the correct extractor implementation based on a document's file extension, keeping that decision out of the Indexer itself.

5. Notes and next steps

- This diagram covers structure only. A sequence diagram for "a query comes in" (SearchEngineAPI → QueryParser → MatchingEngine → RankingAlgorithm → response) is a natural companion piece for your submission.
- Query is shown here as parallel term/operator lists for simplicity. If you implement boolean logic and phrase search through an ANTLR grammar, Query will likely become a small tree of node types (TermNode, AndNode, OrNode, PhraseNode) instead — worth revisiting once your grammar is written.
- File-format extraction (Document.rawText being populated from pdf/docx/etc.) is deliberately left out of this diagram — it is a separate concern from indexing, typically handled by a library such as Apache Tika sitting in front of Indexer.
- Persistence (H2/SQLite, or re-indexing on every restart) is a deployment decision, not a class-design one, and can be added later without changing any class here.